

Énoncés de Problèmes pour Entretien Dev Java – Spring – Kafka – AWS

Énoncés & Corrections Détaillées

Conçu par
Dr. BADR EL KHALYLY

Table des matières

Barème & Coefficients	2
1 Architecture Kafka & Résilience	4
1.1 Partie 1 : Répartition des messages	4
1.1.1 Cas avec clé (key)	4
1.1.2 Cas sans clé (key = null)	5
1.2 Partie 2 : Ordre et offsets	6
1.3 Partie 3 : Réplication et tolérance aux pannes	6
1.4 Partie 4 : Panne d'un broker	7
1.5 Partie 5 : Retry, Relay et DLQ	9
2 Système de traitement de commandes extensible	11
2.1 Partie 1 : Problème initial	11
2.2 Partie 2 : Analyse	12
2.3 Partie 3 : Design Patterns	12
2.3.1 Pattern Strategy – Paiement et Livraison	12
2.3.2 Pattern Factory – Création des stratégies	14
2.3.3 Pattern Decorator – Notifications dynamiques	15
2.3.4 Pattern Chain of Responsibility – Pipeline	17
2.4 Partie 4 : Extension – Promotions	19
3 Spring Bean Lifecycle & Scopes	21
3.1 Partie 1 : Analyse du Scope	21
3.2 Partie 2 : Correction	22
3.3 Partie 3 : Injection et Lifecycle	23
3.4 Partie 4 : Bean Lifecycle	25
3.5 Partie 5 : Problème avancé – Prototype et destruction	28
3.6 Partie 6 : Thread Safety	29
3.7 Partie 7 : Bonus Architecture	30
4 Optimisation d'un système de production (Multithreading Java)	31
4.1 Partie 1 : Introduction du multithreading	31
4.2 Partie 2 : Thread Pool	32
4.3 Partie 3 : Dépendances entre tâches	33
4.4 Partie 4 : CompletableFuture	34
4.5 Partie 5 : Exécution parallèle avancée	35
4.6 Partie 6 : Problèmes de concurrence	38
4.7 Partie 7 : Gestion des performances	39
4.8 Partie 8 : Bonus Architecture	40
5 Conception d'une architecture AWS sécurisée et scalable	41
5.1 Architecture VPC et Réseau	41
5.2 Sécurité – Security Groups	42
5.3 Composants applicatifs	42
5.4 Architecture Microservices	43
5.5 Scalabilité et Haute Disponibilité	44

Barème & Coefficients

N°	Problème	Points	Coef.	Total pondéré
1	Architecture Kafka & Résilience	/20	3	/60
2	Design Patterns Java (Commandes)	/20	3	/60
3	Spring Bean Lifecycle & Scopes	/20	2	/40
4	Multithreading Java	/20	2	/40
5	Architecture AWS	/20	2	/40
Total		12		/240

Détail du barème par problème :

Problème 1 – Kafka (Coef. 3)		
P1.1	Répartition avec clé / sans clé	/4
P1.2	Ordre et offsets	/4
P1.3	Réplication et tolérance aux pannes (leader, follower, ISR)	/4
P1.4	Panne d'un broker et failover	/4
P1.5	Retry, Relay et DLQ	/4

Problème 2 – Design Patterns (Coef. 3)		
P2.1	Analyse des problèmes du code initial	/4
P2.2	Pattern Strategy (paiement + livraison)	/5
P2.3	Pattern Decorator (notifications dynamiques)	/4
P2.4	Chain of Responsibility / Pipeline	/3
P2.5	Extension – Promotions (Factory, composition)	/4

Problème 3 – Spring (Coef. 2)		
P3.1	Analyse du scope (singleton, problème multi-thread)	/3
P3.2	Correction (request, prototype, différences)	/3
P3.3	Injection dynamique (Provider, @Lookup, proxy)	/4
P3.4	Bean Lifecycle (@PostConstruct, @PreDestroy)	/3
P3.5	Prototype destruction + fuites mémoire	/3
P3.6	Thread Safety (ThreadLocal, stateless)	/2
P3.7	Bonus Architecture hexagonale	/2

Problème 4 – Multithreading (Coef. 2)		
P4.1	Analyse de l'approche naïve (threads manuels)	/2
P4.2	Thread Pool (dimensionnement, Fixed vs Cached)	/4
P4.3	Dépendances entre tâches	/2
P4.4	CompletableFuture (chaînage, exceptions)	/4
P4.5	Exécution parallèle (allOf, thenCombine)	/3
P4.6	Problèmes de concurrence (race conditions, solutions)	/3
P4.7	Performances + Bonus architecture	/2

Problème 5 – AWS (Coef. 2)		
P5.1	VPC, Subnets, AZ (architecture réseau)	/5
P5.2	Security Groups (moindre privilège)	/4
P5.3	Composants applicatifs (ECS, ALB, S3, CloudFront, RDS)	/5
P5.4	Architecture microservices + communication	/4
P5.5	Scalabilité et haute disponibilité	/2

Problème 1 : Architecture Kafka & Résilience

Contexte

Une entreprise met en place une architecture basée sur **Apache Kafka** pour gérer des événements de commandes clients.

Infrastructure :

- 4 brokers : B1, B2, B3, B4
- 1 topic nommé `orders`
- 3 partitions : P0, P1, P2
- `replication.factor = 2`

1.1 Partie 1 : Répartition des messages

Un producteur envoie des messages vers le topic `orders`.

1.1.1 Cas avec clé (key)

Les messages envoyés ont une clé correspondant à `customerId`.

Questions

1. Expliquer comment Kafka détermine la partition cible pour chaque message.
2. Si deux messages ont la même clé (`customerId = 123`), seront-ils envoyés dans la même partition ? Dans le même ordre ?
3. Donner un exemple de répartition des messages suivants :
(`key=101, msg=A`), (`key=102, msg=B`), (`key=101, msg=C`), (`key=103, msg=D`)
4. Expliquer le rôle du hash de la clé dans cette distribution.

✓ Correction

1. Kafka utilise la formule : `partition = hash(key) % nombre_partitions`. Le **DefaultPartitioner** applique un hash **murmur2** sur la clé sérialisée, puis effectue un modulo par le nombre de partitions. Cela garantit que la même clé atterrit toujours dans la même partition.
2. Oui. Deux messages avec `customerId = 123` seront envoyés dans la **même partition** (car `hash(123)` est déterministe). L'ordre est **garanti au sein de la partition** : le premier message envoyé aura un offset inférieur, donc sera lu en premier.
3. Exemple de répartition (avec 3 partitions) :
 - `hash(101) % 3 = 0` $\Rightarrow$ P0 reçoit msg A puis msg C
 - `hash(102) % 3 = 1` $\Rightarrow$ P1 reçoit msg B
 - `hash(103) % 3 = 2` $\Rightarrow$ P2 reçoit msg D
4. Le hash de la clé assure une **distribution déterministe et uniforme**. Il transforme n'importe quelle clé (String, int...) en un entier, qui modulo le nombre de partitions

donne l'index de la partition cible. L'algorithme murmur2 offre une bonne répartition statistique, évitant les hotspots (surcharge d'une partition).

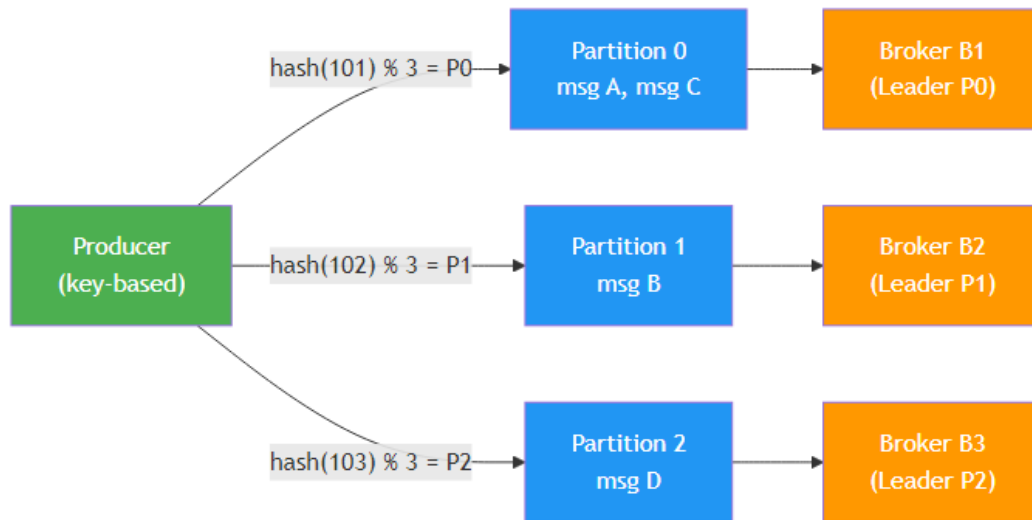


FIGURE 1 – Répartition des messages par clé dans Kafka

1.1.2 Cas sans clé (key = null)

Questions

1. Comment Kafka choisit la partition lorsque la clé est absente ?
2. Quelle stratégie est utilisée (round-robin ou autre selon version) ?
3. Donner un exemple de distribution des messages : (msg=E), (msg=F), (msg=G), (msg=H)
4. Peut-on garantir l'ordre global des messages dans ce cas ? Pourquoi ?

✓ Correction

1. Lorsque la clé est **null**, Kafka n'a pas de critère de hachage. Le producteur choisit alors la partition selon une stratégie configurable.
2.
 - **Kafka < 2.4** : **round-robin** – les messages sont distribués cycliquement sur les partitions.
 - **Kafka ≥ 2.4** : **sticky partitioner** – le producteur envoie un batch complet vers une seule partition avant de passer à la suivante. Cela améliore les performances en réduisant le nombre de requêtes réseau.
3. Exemple en round-robin :
 - msg E → P0
 - msg F → P1
 - msg G → P2
 - msg H → P0

4. **Non**, on ne peut pas garantir l'ordre global. Kafka ne garantit l'ordre que **dans une partition**. Comme les messages sans clé sont dispersés sur différentes partitions, un consommateur lisant plusieurs partitions peut les recevoir dans un ordre différent de l'envoi.

1.2 Partie 2 : Ordre et offsets

Chaque partition maintient ses propres offsets.

Questions

1. Définir ce qu'est un offset dans Kafka.
2. Si les messages M1, M2, M3, M4 sont envoyés dans P1, donner leurs offsets.
3. Kafka garantit-il l'ordre global sur tout le topic ? L'ordre dans une partition ?
4. Si deux consommateurs lisent la même partition dans un même consumer group, que se passe-t-il ?

✓ Correction

1. Un **offset** est un identifiant numérique unique, séquentiel et immutable attribué à chaque message dans une partition. C'est un entier **long** qui commence à 0 et s'incrémente de 1 pour chaque nouveau message. Il sert de position de lecture pour les consommateurs.

2. Offsets dans P1 :

Message	Offset
M1	0
M2	1
M3	2
M4	3

3.

- **Ordre global : NON**. Kafka ne garantit pas l'ordre entre les partitions.
- **Ordre dans une partition : OUI**. Les messages sont lus dans l'ordre exact de leurs offsets.

4. Dans un même **consumer group**, une partition ne peut être assignée qu'à **un seul consommateur**. Si deux consommateurs sont dans le même groupe, le second sera **inactif** pour cette partition (il n'en lira aucun message). Kafka effectue un **rebalancing** pour répartir les partitions équitablement.

1.3 Partie 3 : Réplication et tolérance aux pannes

Le topic `orders` a un `replication.factor = 2`.

Questions

1. Expliquer la notion de leader et follower.
2. Donner un exemple de répartition des leaders et replicas sur les 4 brokers.

3. Que signifie ISR (In-Sync Replica) ?

✓ Correction

1.

- **Leader** : Le broker qui gère toutes les lectures et écritures pour une partition donnée. Il est le point de contact pour les producteurs et consommateurs.
- **Follower** : Un broker qui maintient une copie (réplica) de la partition. Il réplique passivement les données du leader. En cas de panne du leader, un follower ISR peut être promu leader.

2. Exemple de répartition (replication factor = 2) :

Partition	Leader	Follower	Broker libre
P0	B1	B2	B3, B4
P1	B2	B3	B1, B4
P2	B3	B1	B2, B4

Kafka distribue les leaders et followers de manière à équilibrer la charge entre brokers.

3. ISR (In-Sync Replica) désigne l'ensemble des répliques qui sont à jour avec le leader. Un follower est dans l'ISR s'il a répliqué tous les messages du leader dans un délai configurable (`replica.lag.time.max.ms`). Seuls les membres de l'ISR peuvent être élus leader en cas de panne.

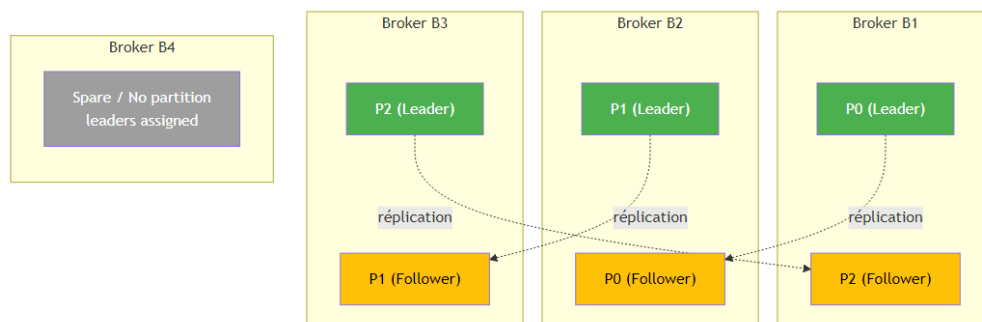


FIGURE 2 – Répartition leaders/followers sur 4 brokers

1.4 Partie 4 : Panne d'un broker

Supposons que le broker B2 tombe en panne.

Questions

1. Que se passe-t-il si B2 était leader d'une partition ? Follower ?
2. Comment Kafka élit un nouveau leader ?
3. Y a-t-il une perte de données possible ? Dans quel cas ?
4. Que se passe-t-il si un message n'a pas été répliqué sur l'ISR ?

✓ Correction

1.

- **B2 était leader** : Le controller Kafka détecte la panne (perte de heartbeat). Il élit un nouveau leader parmi les followers ISR de cette partition. Les clients sont redirigés automatiquement vers le nouveau leader via la mise à jour des métadonnées.
- **B2 était follower** : L'impact est moindre. La partition continue de fonctionner normalement via son leader. L'ISR est réduit (B2 en est retiré). La réplication est temporairement dégradée.

2. Le **Controller** (un broker élu coordinateur du cluster) :

1. Détecte la perte du broker via ZooKeeper / KRaft
2. Consulte la liste ISR de chaque partition affectée
3. Sélectionne le premier follower ISR comme nouveau leader
4. Met à jour les métadonnées du cluster
5. Notifie tous les brokers et clients

3. Perte de données possible si :

- Le producteur utilise `acks=0` ou `acks=1` et le leader tombe avant que le follower n'ait répliqué le message.
- Avec `acks=all` (ou `acks=-1`), le producteur attend la confirmation de tous les ISR $\Rightarrow$ **pas de perte**.
- Si `unclean.leader.election.enable=true`, un follower hors ISR peut devenir leader, causant une perte des messages non répliqués.

4. Si un message n'a pas été répliqué sur l'ISR et que le leader tombe :

- Avec `unclean.leader.election.enable=false` (défaut) : la partition devient **indisponible** jusqu'au retour du leader original.
- Avec `unclean.leader.election.enable=true` : un follower hors ISR est promu, et le **message est perdu**.

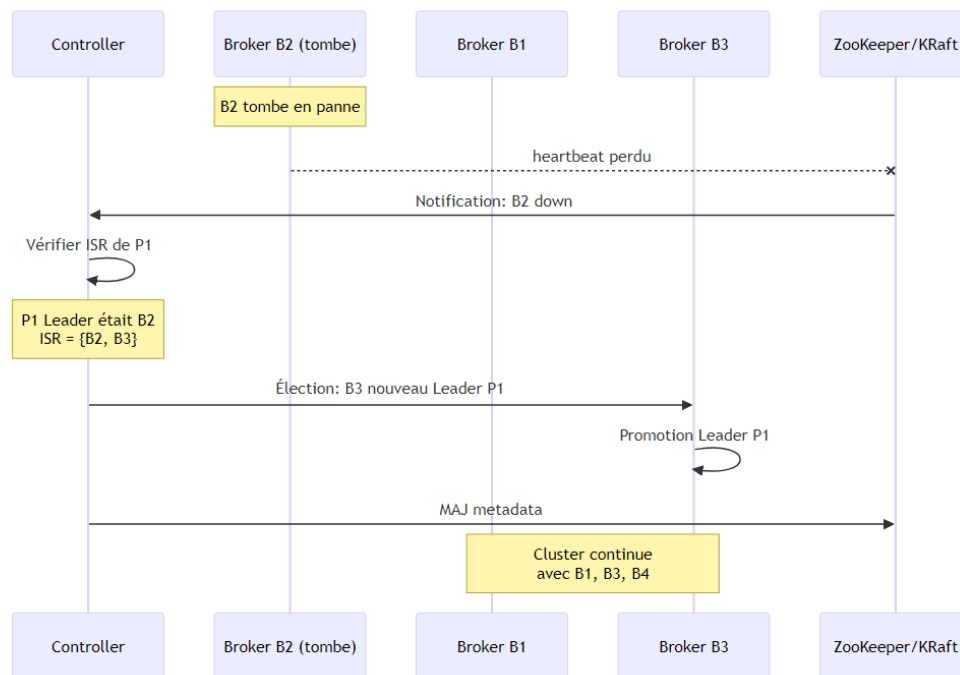


FIGURE 3 – Séquence de failover lors de la panne de B2

1.5 Partie 5 : Retry, Relay et DLQ

L'application consommatrice peut échouer à traiter certains messages.

Questions

1. Expliquer le mécanisme de retry côté consommateur.
2. Qu'est-ce qu'un topic de retry (relay topic) ?
3. Qu'est-ce qu'une Dead Letter Queue (DLQ) ?
4. Proposer une architecture avec `orders`, `orders-retry`, `orders-dlq`.
5. Dans quel cas un message doit être envoyé vers la DLQ ?

✓ Correction

1. Mécanisme de retry : Lorsqu'un consommateur échoue à traiter un message, au lieu de le perdre, il peut :

- Retenter immédiatement (retry local) avec un nombre max de tentatives
- Publier le message dans un **topic de retry** avec un délai (backoff exponentiel)
- Après épuisement des retries, envoyer le message en DLQ

2. Topic de retry (relay topic) : C'est un topic Kafka intermédiaire où sont placés les messages en échec pour être retraités ultérieurement. On peut avoir plusieurs niveaux : `orders-retry-1` (délai 1 min), `orders-retry-2` (délai 5 min), `orders-retry-3` (délai 30 min).

3. Dead Letter Queue (DLQ) : C'est un topic spécial qui stocke les messages **impossibles à traiter** après toutes les tentatives de retry. Ces messages nécessitent une **intervention manuelle** (analyse, correction, rejeu).

4. Voir le diagramme ci-dessous pour l'architecture proposée.

5. Un message est envoyé en DLQ quand :

- Le nombre maximum de retries est atteint
- Le message est **malformé** (erreur de désérialisation)
- Une erreur métier **non récupérable** survient (ex : client inexistant)
- Un **poison pill** : message qui fait crasher le consommateur systématiquement

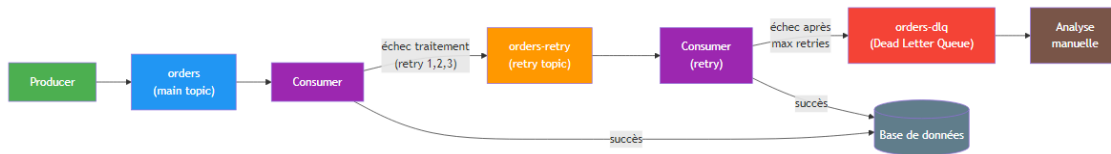


FIGURE 4 – Architecture Retry / DLQ avec Kafka

Problème 2 : Système de traitement de commandes extensible

Contexte

Une entreprise e-commerce souhaite développer un système de traitement de commandes en Java. Chaque commande passe par : **Validation** → **Paielement** → **Livraison** → **Notification**.

Contraintes : plusieurs types de paiement (CB, PayPal, virement), plusieurs modes de livraison (standard, express, international), fonctionnalités optionnelles dynamiques (email, SMS, logging, audit).

2.1 Partie 1 : Problème initial

Listing 1 – Implémentation initiale problématique

```
1 public class OrderService {
2     public void processOrder(Order order, String paymentType,
3                               String deliveryType) {
4         // Validation
5         if (order == null) {
6             throw new RuntimeException("Invalid order");
7         }
8         // Paiement
9         if ("CARD".equals(paymentType)) {
10             // logique carte
11         } else if ("PAYPAL".equals(paymentType)) {
12             // logique paypal
13         } else if ("BANK".equals(paymentType)) {
14             // logique virement
15         }
16         // Livraison
17         if ("STANDARD".equals(deliveryType)) {
18             // logique standard
19         } else if ("EXPRESS".equals(deliveryType)) {
20             // logique express
21         }
22         // Notification
23         System.out.println("Send email");
24     }
25 }
```

2.2 Partie 2 : Analyse

Questions

1. Quels sont les problèmes de conception dans ce code ?
2. Pourquoi cette implémentation ne respecte pas le principe Open/Closed ?
3. Quels sont les impacts si on ajoute un nouveau moyen de paiement ou un nouveau type de livraison ?

✓ Correction

1. Problèmes de conception :

- **Violation du SRP** (Single Responsibility) : la classe gère validation, paiement, livraison et notification.
- **Couplage fort** : la logique métier est mélangée dans une seule méthode.
- **Chaînes if/else** : code fragile, difficulté à tester unitairement.
- **Utilisation de String magiques** : pas de typage fort pour `paymentType` et `deliveryType`.
- **Notification en dur** : impossible d'ajouter SMS/Push sans modifier le code.

2. Le **principe Open/Closed** stipule qu'une classe doit être **ouverte à l'extension** mais **fermée à la modification**. Ici, chaque ajout de paiement/livraison nécessite de **modifier** la méthode `processOrder`, ce qui viole ce principe.

3. Impacts d'un ajout :

- Modification du code existant (risque de régression)
- Ajout d'un nouveau `else if` (complexité croissante)
- Recompilation et redéploiement de toute la classe
- Tests existants à rejouer intégralement

2.3 Partie 3 : Design Patterns

Questions

1. Comment gérer les différents types de paiement ?
2. Comment rendre la livraison interchangeable ?
3. Comment ajouter dynamiquement des comportements (email, SMS, logging...) ?
4. Comment structurer le pipeline de traitement ?

2.3.1 Pattern Strategy – Paiement et Livraison

✓ Correction

Le pattern **Strategy** permet d'encapsuler des algorithmes interchangeables derrière une interface commune.

```
1 public interface PaymentStrategy {
2     void pay(Order order);
3 }
4
5 public class CardPayment implements PaymentStrategy {
6     @Override
7     public void pay(Order order) {
8         System.out.println("Paielement par carte : " + order.getTotal
9             ());
10    }
11 }
12
13 public class PayPalPayment implements PaymentStrategy {
14     @Override
15     public void pay(Order order) {
16         System.out.println("Paielement PayPal : " + order.getTotal())
17             ;
18    }
19 }
20
21 public class BankTransferPayment implements PaymentStrategy {
22     @Override
23     public void pay(Order order) {
24         System.out.println("Virement bancaire : " + order.getTotal
25             ());
26    }
27 }
```

Listing 3 – Pattern Strategy pour la livraison

```
1 public interface DeliveryStrategy {
2     void deliver(Order order);
3 }
4
5 public class StandardDelivery implements DeliveryStrategy {
6     @Override
7     public void deliver(Order order) {
8         System.out.println("Livraison standard : 5-7 jours");
9    }
10 }
11
12 public class ExpressDelivery implements DeliveryStrategy {
13     @Override
14     public void deliver(Order order) {
15         System.out.println("Livraison express : 24h");
16    }
17 }
18
19 public class InternationalDelivery implements DeliveryStrategy {
20     @Override
21     public void deliver(Order order) {
```

```

22         System.out.println("Livraison internationale : 10-15 jours"
23                               );
24     }

```

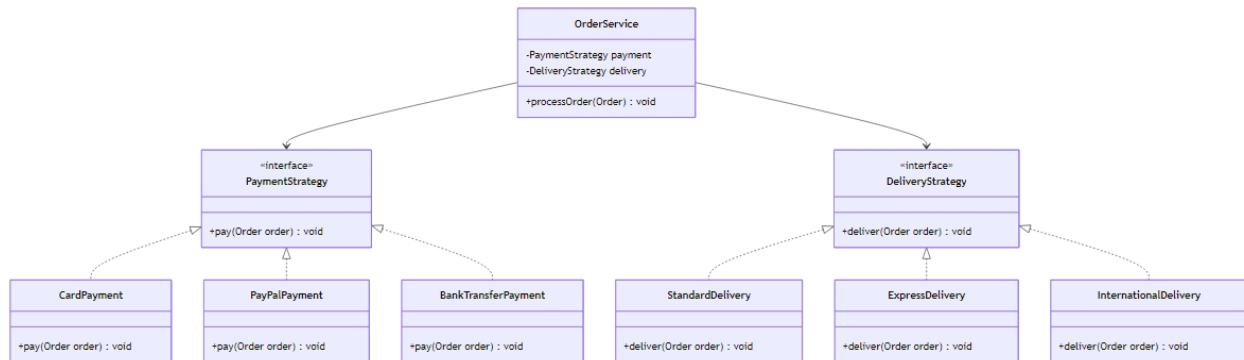


FIGURE 5 – Diagramme de classes – Pattern Strategy

2.3.2 Pattern Factory – Création des stratégies

✓ Correction

Le pattern **Factory** centralise la création des objets et élimine les `if/else` dans le code client.

Listing 4 – Factory pour les stratégies de paiement

```

1  public class PaymentFactory {
2      private static final Map<String, Supplier<PaymentStrategy>>
3          registry
4          = new HashMap<>();
5
6      static {
7          registry.put("CARD", CardPayment::new);
8          registry.put("PAYPAL", PayPalPayment::new);
9          registry.put("BANK", BankTransferPayment::new);
10     }
11
12     public static PaymentStrategy create(String type) {
13         Supplier<PaymentStrategy> supplier = registry.get(type);
14         if (supplier == null) {
15             throw new IllegalArgumentException(
16                 "Type de paiement inconnu : " + type);
17         }
18         return supplier.get();
19     }
20
21     // Extensible : ajouter un nouveau type sans modifier le code
22     public static void register(String type,
23                               Supplier<PaymentStrategy> supplier
24     ) {

```

```

23     registry.put(type, supplier);
24 }
25 }

```

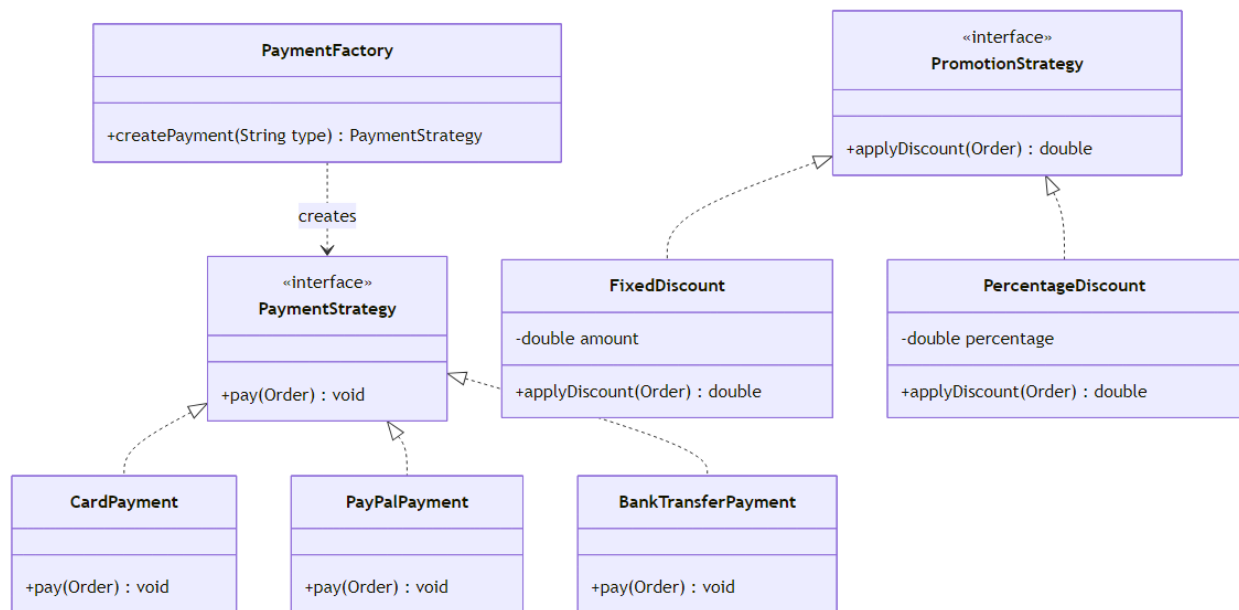


FIGURE 6 – Diagramme de classes – Pattern Factory et Promotion

2.3.3 Pattern Decorator – Notifications dynamiques

✓ Correction

Le pattern **Decorator** permet d'ajouter dynamiquement des comportements sans modifier les classes existantes.

Listing 5 – Pattern Decorator pour les notifications

```

1  public interface Notifier {
2      void send(String message);
3  }
4
5  public class BaseNotifier implements Notifier {
6      @Override
7      public void send(String message) {
8          System.out.println("Notification de base : " + message);
9      }
10 }
11
12 public abstract class NotifierDecorator implements Notifier {
13     protected Notifier wrapped;
14
15     public NotifierDecorator(Notifier wrapped) {
16         this.wrapped = wrapped;
17     }
18 }

```



```

19     @Override
20     public void send(String message) {
21         wrapped.send(message);
22     }
23 }
24
25 public class EmailDecorator extends NotifierDecorator {
26     public EmailDecorator(Notifier wrapped) {
27         super(wrapped);
28     }
29     @Override
30     public void send(String message) {
31         super.send(message);
32         System.out.println("+ Email envoye : " + message);
33     }
34 }
35
36 public class SmsDecorator extends NotifierDecorator {
37     public SmsDecorator(Notifier wrapped) {
38         super(wrapped);
39     }
40     @Override
41     public void send(String message) {
42         super.send(message);
43         System.out.println("+ SMS envoye : " + message);
44     }
45 }

```

Utilisation :

Listing 6 – Composition dynamique des décorateurs

```

1  Notifier notifier = new LoggingDecorator(
2                          new SmsDecorator(
3                              new EmailDecorator(
4                                  new BaseNotifier())));
5  notifier.send("Commande confirmee");
6  // Output :
7  // Notification de base : Commande confirmee
8  // + Email envoye : Commande confirmee
9  // + SMS envoye : Commande confirmee
10 // [LOG] Commande confirmee

```

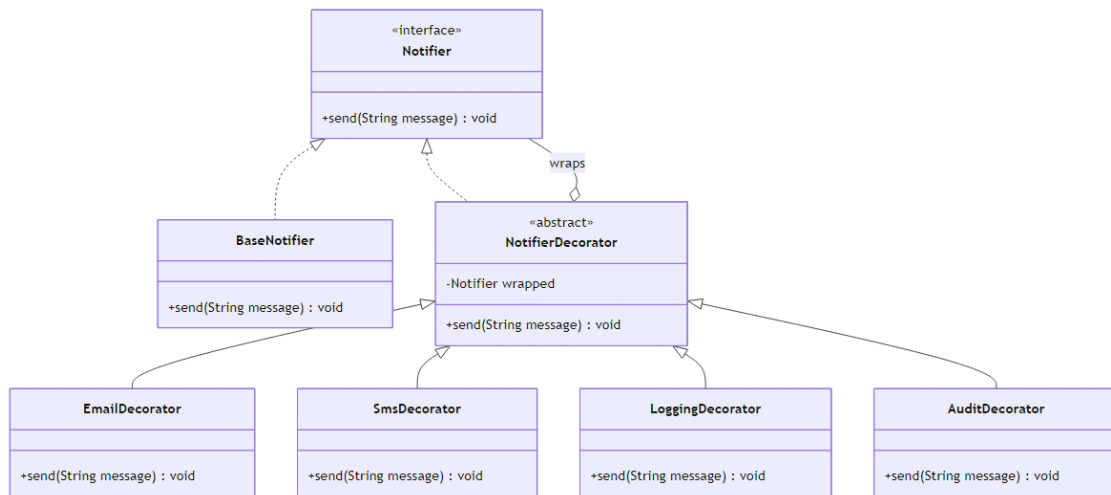


FIGURE 7 – Diagramme de classes – Pattern Decorator

2.3.4 Pattern Chain of Responsibility – Pipeline

✓ Correction

Le pattern **Chain of Responsibility** (ou Pipeline) permet de chaîner des handlers, chacun traitant une étape du processus.

Listing 7 – Pipeline avec Chain of Responsibility

```

1  public interface OrderHandler {
2      void setNext(OrderHandler next);
3      void handle(Order order);
4  }
5
6  public abstract class AbstractOrderHandler implements OrderHandler
7  {
8      private OrderHandler next;
9
10     @Override
11     public void setNext(OrderHandler next) {
12         this.next = next;
13     }
14
15     protected void passToNext(Order order) {
16         if (next != null) {
17             next.handle(order);
18         }
19     }
20
21     public class ValidationHandler extends AbstractOrderHandler {
22         @Override
23         public void handle(Order order) {
24             if (order == null || order.getItems().isEmpty()) {
25                 throw new RuntimeException("Commande invalide");

```

```

26     }
27     System.out.println("Validation OK");
28     passToNext(order);
29 }
30 }
31
32 public class PaymentHandler extends AbstractOrderHandler {
33     private final PaymentStrategy strategy;
34
35     public PaymentHandler(PaymentStrategy strategy) {
36         this.strategy = strategy;
37     }
38
39     @Override
40     public void handle(Order order) {
41         strategy.pay(order);
42         passToNext(order);
43     }
44 }

```

Construction du pipeline :

Listing 8 – Construction du pipeline complet

```

1 ValidationHandler validation = new ValidationHandler();
2 PaymentHandler payment = new PaymentHandler(new CardPayment());
3 DeliveryHandler delivery = new DeliveryHandler(new ExpressDelivery
4     ());
5 NotificationHandler notification = new NotificationHandler(notifier
6     );
7
8 validation.setNext(payment);
9 payment.setNext(delivery);
10 delivery.setNext(notification);
11
12 // Lancer le pipeline
13 validation.handle(order);

```

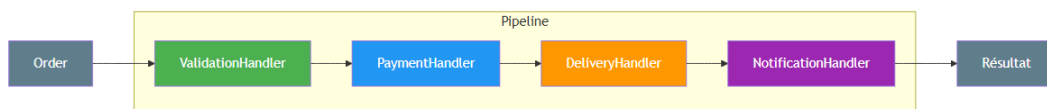


FIGURE 8 – Pipeline de traitement – Chain of Responsibility

2.4 Partie 4 : Extension – Promotions

Questions

1. Comment intégrer un système de promotion (réduction fixe, pourcentage) ?
2. Comment limiter certaines promotions à certains types de clients ?
3. Comment éviter de complexifier le code existant ?

✓ Correction

On utilise à nouveau le **pattern Strategy** pour les promotions :

Listing 9 – Strategy pour les promotions

```
1 public interface PromotionStrategy {
2     double applyDiscount(Order order);
3 }
4
5 public class FixedDiscount implements PromotionStrategy {
6     private final double amount;
7
8     public FixedDiscount(double amount) {
9         this.amount = amount;
10    }
11
12    @Override
13    public double applyDiscount(Order order) {
14        return Math.max(0, order.getTotal() - amount);
15    }
16 }
17
18 public class PercentageDiscount implements PromotionStrategy {
19     private final double percentage;
20
21     public PercentageDiscount(double percentage) {
22         this.percentage = percentage;
23    }
24
25    @Override
26    public double applyDiscount(Order order) {
27        return order.getTotal() * (1 - percentage / 100);
28    }
29 }
```

Pour les règles liées aux types de clients, on ajoute un **Decorator conditionnel** :

Listing 10 – Promotion conditionnelle par type de client

```
1 public class CustomerTypePromotion implements
2     PromotionStrategy {
3     private final PromotionStrategy delegate;
4     private final CustomerType requiredType;
```

```
4
5     public CustomerTypePromotion(PromotionStrategy delegate,
6                                   CustomerType requiredType) {
7         this.delegate = delegate;
8         this.requiredType = requiredType;
9     }
10
11     @Override
12     public double applyDiscount(Order order) {
13         if (order.getCustomer().getType() == requiredType) {
14             return delegate.applyDiscount(order);
15         }
16         return order.getTotal(); // pas de reduction
17     }
18 }
19
20 // Utilisation :
21 PromotionStrategy vipPromo = new CustomerTypePromotion(
22     new PercentageDiscount(20), CustomerType.VIP);
```

Intégration dans le pipeline : On ajoute un `PromotionHandler` dans la chaîne, entre la validation et le paiement, **sans modifier les handlers existants**.

Problème 3 : Spring Bean Lifecycle & Scopes

Contexte

Une application Java Spring Boot gère un système de notifications (Email, SMS, Push). Le système repose sur plusieurs beans Spring injectés via IoC/DI :

- `NotificationService` (service principal, singleton)
- `EmailSender`, `SmsSender`
- `NotificationContext` (données runtime liées à une requête)

Code initial :

Listing 11 – Implémentation initiale avec problème de scope

```
1  @Service
2  public class NotificationService {
3      @Autowired
4      private NotificationContext context;
5
6      public void send(String message) {
7          System.out.println("User: " + context.getUser());
8          System.out.println("Message: " + message);
9      }
10 }
11
12 @Component
13 @Scope("singleton")
14 public class NotificationContext {
15     private String user;
16
17     public String getUser() { return user; }
18     public void setUser(String user) { this.user = user; }
19 }
```

3.1 Partie 1 : Analyse du Scope

Questions

1. Quel est le scope par défaut d'un bean Spring ?
2. `NotificationContext` est en singleton : est-ce adapté ?
3. Quels problèmes cela engendre dans une application web multi-utilisateurs ?
4. Décrire un scénario réel où ce design provoque un bug critique.

✓ Correction

1. Le scope par défaut est **singleton** : Spring crée une seule instance du bean, partagée par tout le contexte applicatif.

2. Non, ce n'est pas adapté. NotificationContext contient un état mutable (`user`) lié à une requête. Un singleton partage cette instance entre **tous les threads/requêtes**.

3. Problèmes :

- **Race condition** : plusieurs threads modifient `user` simultanément
- **Fuite de données** : un utilisateur voit les données d'un autre
- **Comportement non déterministe** : résultats différents selon l'ordre d'exécution

4. Scénario de bug critique :

1. Thread A (User Chouaib) appelle `context.setUser("Chouaib")`
2. Thread B (User Ali) appelle `context.setUser("Ali")` ← écrase Chouaib !
3. Thread A appelle `send("Hello")`
4. `context.getUser()` retourne "Ali" au lieu de "Chouaib"
5. Chouaib reçoit la notification destinée à Ali : **violation de confidentialité**

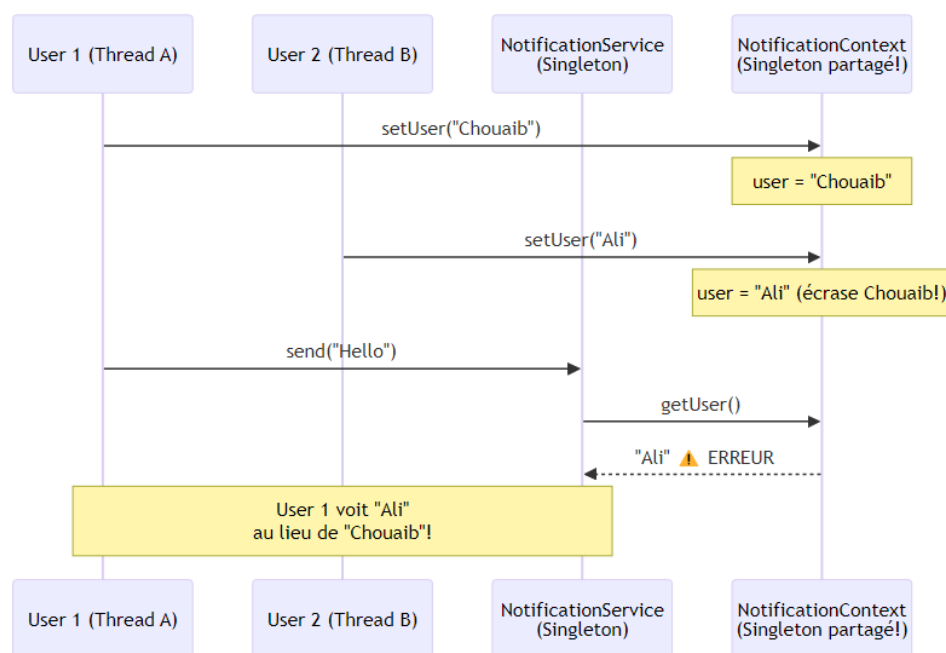


FIGURE 9 – Problème de concurrence avec un singleton mutable

3.2 Partie 2 : Correction

Questions

1. Quel scope serait plus adapté pour NotificationContext ?
2. Quelle est la différence entre **prototype**, **request** et **session** ?
3. Si on passe en **prototype**, est-ce suffisant ? Pourquoi Spring peut injecter la même instance ?

✓ Correction

1. Le scope **request** est le plus adapté dans une application web. Il crée une instance par requête HTTP, assurant l'isolation entre utilisateurs.

2. Différences :

Scope	Description
prototype	Nouvelle instance à chaque injection (chaque appel à <code>getBean()</code>). Spring ne gère pas le cycle de vie après création.
request	Une instance par requête HTTP . Détruite à la fin de la requête. Nécessite un contexte web.
session	Une instance par session HTTP . Persiste entre les requêtes d'un même utilisateur.

3. **Non, prototype seul ne suffit pas!** Le problème est le suivant : `NotificationService` est un **singleton**. L'injection de `NotificationContext` a lieu **une seule fois**, lors de la création du singleton. Même si `NotificationContext` est **prototype**, la même instance sera utilisée pour toute la durée de vie du singleton.

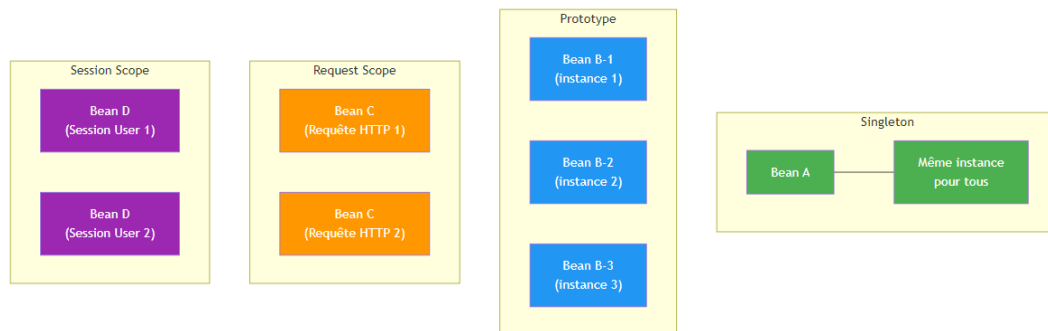


FIGURE 10 – Comparaison des scopes Spring

3.3 Partie 3 : Injection et Lifecycle

Questions

1. Expliquer pourquoi `NotificationService` reçoit toujours la même instance.
2. Comment forcer Spring à injecter une nouvelle instance à chaque appel ?
3. Proposer au moins deux solutions.

✓ Correction

1. Le singleton `NotificationService` est créé une seule fois au démarrage. Spring résout ses dépendances **à ce moment-là** : il injecte une instance de `NotificationContext` qui reste figée dans le champ `context`. Les appels ultérieurs à `send()` utilisent toujours cette même référence.

2. & 3. Solutions :

Solution 1 : `ObjectFactory` / `ObjectProvider`

Listing 12 – Injection via `ObjectProvider`


```

1  @Service
2  public class NotificationService {
3      @Autowired
4      private ObjectProvider<NotificationContext>
        contextProvider;
5
6      public void send(String message) {
7          NotificationContext ctx = contextProvider.getObject();
8          System.out.println("User: " + ctx.getUser());
9      }
10 }

```

Solution 2 : javax.inject.Provider

Listing 13 – Injection via Provider

```

1  @Service
2  public class NotificationService {
3      @Autowired
4      private Provider<NotificationContext> contextProvider;
5
6      public void send(String message) {
7          NotificationContext ctx = contextProvider.get();
8          System.out.println("User: " + ctx.getUser());
9      }
10 }

```

Solution 3 : @Lookup

Listing 14 – Injection via @Lookup

```

1  @Service
2  public abstract class NotificationService {
3
4      public void send(String message) {
5          NotificationContext ctx = createContext();
6          System.out.println("User: " + ctx.getUser());
7      }
8
9      @Lookup
10     protected abstract NotificationContext createContext();
11 }

```

Solution 4 : Scope request avec proxy

Listing 15 – Scope request avec proxy

```

1  @Component
2  @Scope(value = "request", proxyMode = ScopedProxyMode.
    TARGET_CLASS)
3  public class NotificationContext {
4      private String user;

```

```
5 // getters, setters
6 }
```

Avec `proxyMode`, Spring injecte un **proxy** qui délègue vers l'instance correcte pour chaque requête.

3.4 Partie 4 : Bean Lifecycle

Listing 16 – Bean avec callbacks de lifecycle

```
1 @Component
2 public class EmailSender {
3     @PostConstruct
4     public void init() {
5         System.out.println("Init EmailSender");
6     }
7
8     @PreDestroy
9     public void destroy() {
10        System.out.println("Destroy EmailSender");
11    }
12 }
```

Questions

1. Décrire les différentes phases du cycle de vie d'un bean Spring.
2. Quand est appelée `@PostConstruct` ?
3. Est-ce que `@PreDestroy` est appelée pour un bean **prototype** ? Pourquoi ?

✓ Correction

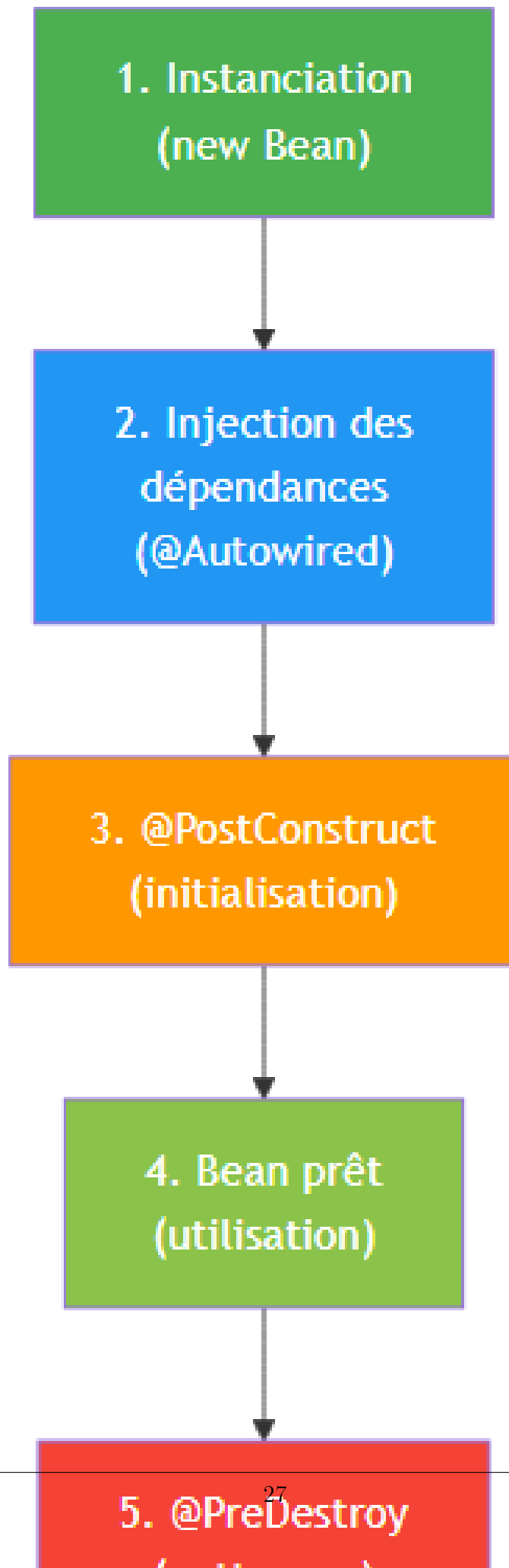
1. Phases du cycle de vie :

1. **Instanciation** : Spring crée l'objet (via constructeur ou factory method)
2. **Injection des dépendances** : Spring injecte les dépendances (`@Autowired`, constructeur, setter)
3. **Post-processing** : les `BeanPostProcessor` sont appliqués (`postProcessBeforeInitialization`)
4. **Initialisation** : appel de `@PostConstruct`, puis `InitializingBean.afterPropertiesSet()`, puis `init-method`
5. **Post-processing (après init)** : `postProcessAfterInitialization` (création de proxies AOP)
6. **Bean prêt** : le bean est utilisable par l'application
7. **Destruction** : `@PreDestroy`, puis `DisposableBean.destroy()`, puis `destroy-method`

2. `@PostConstruct` est appelée **après l'injection des dépendances** et **avant** que le bean ne soit mis à disposition. C'est l'endroit idéal pour l'initialisation qui dépend des

dépendances injectées.

3. Non, `@PreDestroy` n'est **PAS appelée** pour les beans **prototype**. Spring ne gère pas le cycle de vie complet des beans prototype : après création et initialisation, Spring "oublie" le bean. C'est au **code client** de le détruire explicitement.



3.5 Partie 5 : Problème avancé – Prototype et destruction

Questions

1. Pourquoi `@PreDestroy` n'est jamais appelé pour certains beans ?
2. Qui est responsable de la destruction des beans prototype ?
3. Proposer une solution propre pour gérer leur cycle de vie.

✓ Correction

1. Le container Spring ne garde pas de référence vers les beans **prototype** après leur création. Il ne peut donc pas appeler `@PreDestroy` lors de la fermeture du contexte. Si ces beans acquièrent des ressources (connexions, fichiers), elles ne sont jamais libérées ⇒ **fuites mémoire**.

2. Le **code client** (celui qui reçoit le bean) est responsable d'appeler les méthodes de nettoyage.

3. Solutions :

a) Implémenter `DestructionAwareBeanPostProcessor` custom :

```
1  @Component
2  public class PrototypeBeanTracker
3      implements DestructionAwareBeanPostProcessor,
4                  DisposableBean {
5
6      private final List<Object> prototypeBeans =
7          Collections.synchronizedList(new ArrayList<>());
8
9      @Override
10     public Object postProcessAfterInitialization(
11         Object bean, String beanName) {
12         // Tracker les beans prototype
13         if (isPrototype(bean)) {
14             prototypeBeans.add(bean);
15         }
16         return bean;
17     }
18
19     @Override
20     public void destroy() {
21         for (Object bean : prototypeBeans) {
22             if (bean instanceof DisposableBean) {
23                 ((DisposableBean) bean).destroy();
24             }
25         }
26         prototypeBeans.clear();
27     }
```

b) Utiliser `try-with-resources` si le bean implémente `AutoCloseable`.

c) Préférer le scope `request` dans un contexte web, car Spring gère sa destruction

automatiquement.

3.6 Partie 6 : Thread Safety

Listing 17 – Utilisation multi-thread problématique

```

1 Runnable task = () -> {
2     context.setUser(Thread.currentThread().getName());
3     notificationService.send("Hello");
4 };

```

Questions

1. Quels problèmes peuvent apparaître ?
2. Ce bean est-il thread-safe ?
3. Proposer une solution adaptée dans un contexte multi-thread.

✓ Correction

1. Problèmes :

- **Race condition** : le champ `user` est écrit par un thread et lu par un autre
- **Visibilité** : sans `volatile` ou synchronisation, un thread peut ne pas voir l'écriture d'un autre
- **Interleaving** : entre `setUser()` et `send()`, un autre thread peut appeler `setUser()`

2. Non, le bean n'est **pas thread-safe**. Un singleton avec état mutable sans synchronisation est par définition non thread-safe.

3. Solutions :

a) ThreadLocal :

```

1 @Component
2 public class NotificationContext {
3     private final ThreadLocal<String> user = new ThreadLocal
4         <>();
5
6     public String getUser() { return user.get(); }
7     public void setUser(String u) { user.set(u); }
8     public void clear() { user.remove(); }
9 }

```

b) Design stateless (passer le contexte en paramètre) :

```

1 @Service
2 public class NotificationService {
3     public void send(String user, String message) {
4         System.out.println("User: " + user);
5         System.out.println("Message: " + message);
6     }
7 }

```

```
6     }  
7 }
```

c) **Scope request** avec proxy (dans un contexte web).

Recommandation : Privilégier le **design stateless**. Les beans Spring doivent idéalement ne pas contenir d'état mutable.

3.7 Partie 7 : Bonus Architecture

Questions

1. Comment intégrer ce problème dans une architecture hexagonale ?
2. Où placer `NotificationContext` (domain, infra, application) ?
3. Comment éviter l'usage de state mutable dans Spring ?

✓ Correction

1. Architecture hexagonale :

- **Domain** : contient les interfaces (ports) comme `NotificationPort`
- **Application** : orchestre l'envoi via un `use case` (`SendNotificationUseCase`)
- **Infrastructure** : implémente les adapters (`EmailAdapter`, `SmsAdapter`)

2. `NotificationContext` devrait être dans la couche **Application** (ou en tant que DTO dans les ports d'entrée). Il ne fait pas partie du domaine métier, mais transporte des données de contexte d'exécution. L'infrastructure ne devrait pas en dépendre directement.

3. Éviter le state mutable :

- **Beans stateless** : ne pas stocker d'état dans les services
- **Passer le contexte en paramètre** des méthodes
- **Objets immutables** : utiliser des records Java ou des classes finales avec champs `final`
- **DTO/Value Objects** : transporter les données entre couches sans état mutable

Problème 4 : Optimisation d'un système de production (Multithreading Java)

Contexte

Une usine intelligente (Smart Factory) gère une chaîne de production. Chaque commande déclenche : **Vérification matières premières** → **Assemblage** → **Contrôle qualité** → **Emballage** → **Notification**.

L'application est en Java, chaque étape est coûteuse (I/O + CPU), et le système doit traiter plusieurs commandes en parallèle.

Code initial :

Listing 18 – Implémentation séquentielle

```

1 public class ProductionService {
2     public void processOrder(Order order) {
3         checkRawMaterials(order);
4         assemble(order);
5         qualityCheck(order);
6         packaging(order);
7         notifyClient(order);
8     }
9 }
```

4.1 Partie 1 : Introduction du multithreading

Listing 19 – Tentative naïve avec threads

```

1 public void processOrder(Order order) {
2     Thread t1 = new Thread(() -> checkRawMaterials(order));
3     Thread t2 = new Thread(() -> assemble(order));
4     t1.start();
5     t2.start();
6 }
```

Questions

1. Quels sont les problèmes de cette approche ?
2. Que se passe-t-il avec 1000 commandes simultanées ?
3. Pourquoi cette solution n'est pas adaptée en production ?

✓ Correction

1. Problèmes :

— Pas de respect des dépendances : `assemble()` dépend de

`checkRawMaterials()`, mais les deux sont lancées en parallèle.

- **Pas de synchronisation** : le thread principal ne sait pas quand les threads fils terminent (pas de `join()`).
- **Pas de gestion d'erreurs** : une exception dans un thread est silencieusement perdue.
- **Création non contrôlée de threads** : chaque commande crée 2 nouveaux threads.

2. Avec 1000 commandes : **2000 threads créés** simultanément. Chaque thread consomme environ 1 Mo de stack $\Rightarrow$ **2 Go de mémoire** juste pour les stacks. Risque de `OutOfMemoryError`, surcharge du scheduler OS, et **context switching** massif réduisant les performances.

3. En production, on ne crée jamais de threads manuellement car :

- Pas de contrôle sur le nombre de threads actifs
- Pas de réutilisation des threads (coût de création/destruction)
- Pas de file d'attente pour les tâches en excès
- Pas de politique de rejet configurable

4.2 Partie 2 : Thread Pool

Listing 20 – Introduction d'un `ExecutorService`

```
1 ExecutorService executor = Executors.newFixedThreadPool(4);
```

Questions

1. Quel est l'intérêt d'un thread pool ?
2. Différence entre `newFixedThreadPool` et `newCachedThreadPool` ?
3. Comment dimensionner pour CPU-bound vs I/O-bound ?
4. Risques d'un mauvais dimensionnement ?

✓ Correction

1. Un **thread pool** :

- **Réutilise** les threads au lieu d'en créer de nouveaux
- **Limite** le nombre de threads actifs
- Fournit une **file d'attente** pour les tâches en excès
- Offre des **politiques de rejet** configurables
- Améliore la **prévisibilité** des performances

2. **Comparaison** :

	<code>newFixedThreadPool(n)</code>	<code>newCachedThreadPool()</code>
Taille	Fixe (n threads)	Variable (0 à <code>Integer.MAX_VALUE</code>)
File d'attente	Illimitée (<code>LinkedBlockingQueue</code>)	Aucune (<code>SynchronousQueue</code>)
Threads inactifs	Conservés	Supprimés après 60s
Risque	OOM si file saturée	Création de trop de threads
Cas d'usage	Charge prévisible	Beaucoup de tâches courtes

3. Dimensionnement :

- **CPU-bound** : nombre de threads = nombre de cœurs (ou cœurs + 1). Au-delà, le context switching dégrade les performances.
- **I/O-bound** : nombre de threads = nombre de cœurs $\times$ (1 + temps_attente / temps_calcul). Comme les threads passent du temps bloqués en I/O, on peut en avoir davantage.

4. Risques :

- **Sous-dimensionné** : file d'attente qui grossit, latence élevée, tâches rejetées
- **Sur-dimensionné** : gaspillage mémoire, context switching excessif, contention sur les ressources partagées

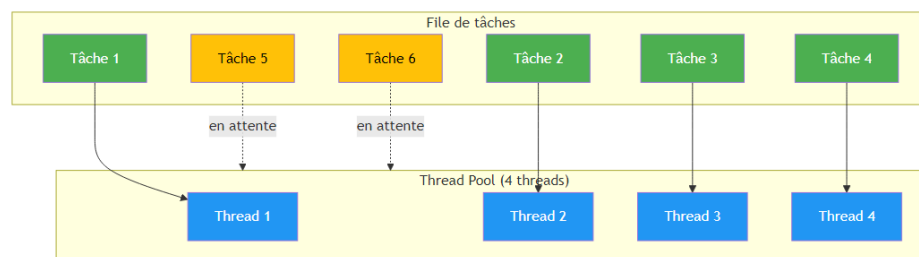


FIGURE 12 – Fonctionnement d'un Thread Pool

4.3 Partie 3 : Dépendances entre tâches

Questions

1. Peut-on paralléliser certaines étapes ?
2. Identifier les parties parallélisables et séquentielles.
3. Proposer une stratégie d'exécution optimale.

✓ Correction

1. Dans le pipeline actuel, toutes les étapes sont **séquentiellement dépendantes** :

`check` → `assemble` → `qualityCheck` → `packaging` → `notify`

On ne peut pas paralléliser au sein d'une commande unique. **Cependant**, on peut paralléliser le traitement de **plusieurs commandes indépendantes** simultanément.

- 2.

- **Parallélisable** : le traitement de différentes commandes (Order A et Order B peuvent être traitées en parallèle)
- **Séquentiel** : les étapes au sein d'une même commande (l'assemblage dépend de la vérification des matières premières)

3. Stratégie optimale : Utiliser un `ExecutorService` pour soumettre chaque commande comme une tâche complète, traitée séquentiellement en interne, mais en parallèle avec les autres commandes.

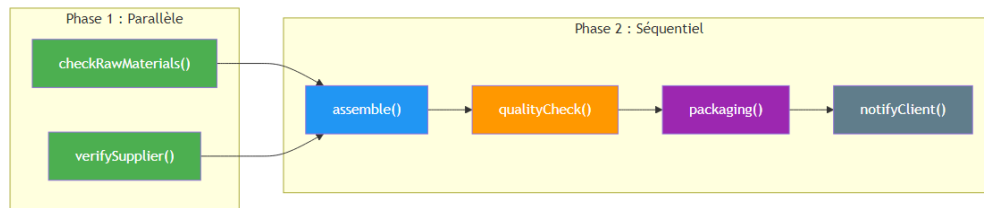


FIGURE 13 – Phases parallélisables vs séquentielles

4.4 Partie 4 : CompletableFuture

Listing 21 – Pipeline avec CompletableFuture

```

1 CompletableFuture<Void> future =
2   CompletableFuture.runAsync(() -> checkRawMaterials(order),
3     executor)
4     .thenRunAsync(() -> assemble(order), executor)
5     .thenRunAsync(() -> qualityCheck(order), executor)
6     .thenRunAsync(() -> packaging(order), executor)
7     .thenRunAsync(() -> notifyClient(order), executor);
  
```

Questions

1. Expliquer le fonctionnement de `CompletableFuture`.
2. Différence entre `runAsync` et `supplyAsync` ?
3. Que permet `thenRunAsync` ?
4. Comment gérer les exceptions ?

✓ Correction

1. `CompletableFuture` est une implémentation de `Future` qui :
 - Permet de **chaîner** des opérations asynchrones
 - Supporte la **composition** de tâches (séquentielles et parallèles)
 - Gère les **exceptions** de manière déclarative
 - Peut être complétée **manuellement** ou par calcul asynchrone
 - S'exécute par défaut sur le `ForkJoinPool.commonPool()`
2. **Différences** :

	runAsync	supplyAsync
Type	CompletableFuture<Void>	CompletableFuture<T>
Paramètre	Runnable	Supplier<T>
Retour	Aucun (void)	Produit une valeur
Usage	Actions sans résultat	Actions avec résultat

3. **thenRunAsync** exécute une action **après** la complétion de l'étape précédente, de manière **asynchrone** (potentiellement sur un autre thread). La version sans **Async** (**thenRun**) s'exécute sur le thread qui a complété l'étape précédente.

4. Gestion des exceptions :

Listing 22 – Gestion des exceptions avec CompletableFuture

```

1  CompletableFuture<Void> future =
2      CompletableFuture.runAsync(() -> checkRawMaterials(order),
3          executor)
4          .thenRunAsync(() -> assemble(order), executor)
5          .thenRunAsync(() -> qualityCheck(order), executor)
6          .exceptionally(ex -> {
7              // Appele si une etape echoue
8              System.err.println("Erreur : " + ex.getMessage());
9              return null;
10         })
11         .handle((result, ex) -> {
12             // Appele dans TOUS les cas (succes ou echec)
13             if (ex != null) {
14                 System.err.println("Echec : " + ex.getMessage());
15             } else {
16                 System.out.println("Succes !");
17             }
18             return result;
19         });

```

4.5 Partie 5 : Exécution parallèle avancée

On ajoute une contrainte : **checkRawMaterials()** et **verifySupplier()** peuvent être exécutées en parallèle.

Questions

1. Comment exécuter ces deux tâches en parallèle ?
2. Comment synchroniser avant de passer à **assemble()** ?
3. Quel opérateur utiliser (**thenCombine**, **allOf**) ?

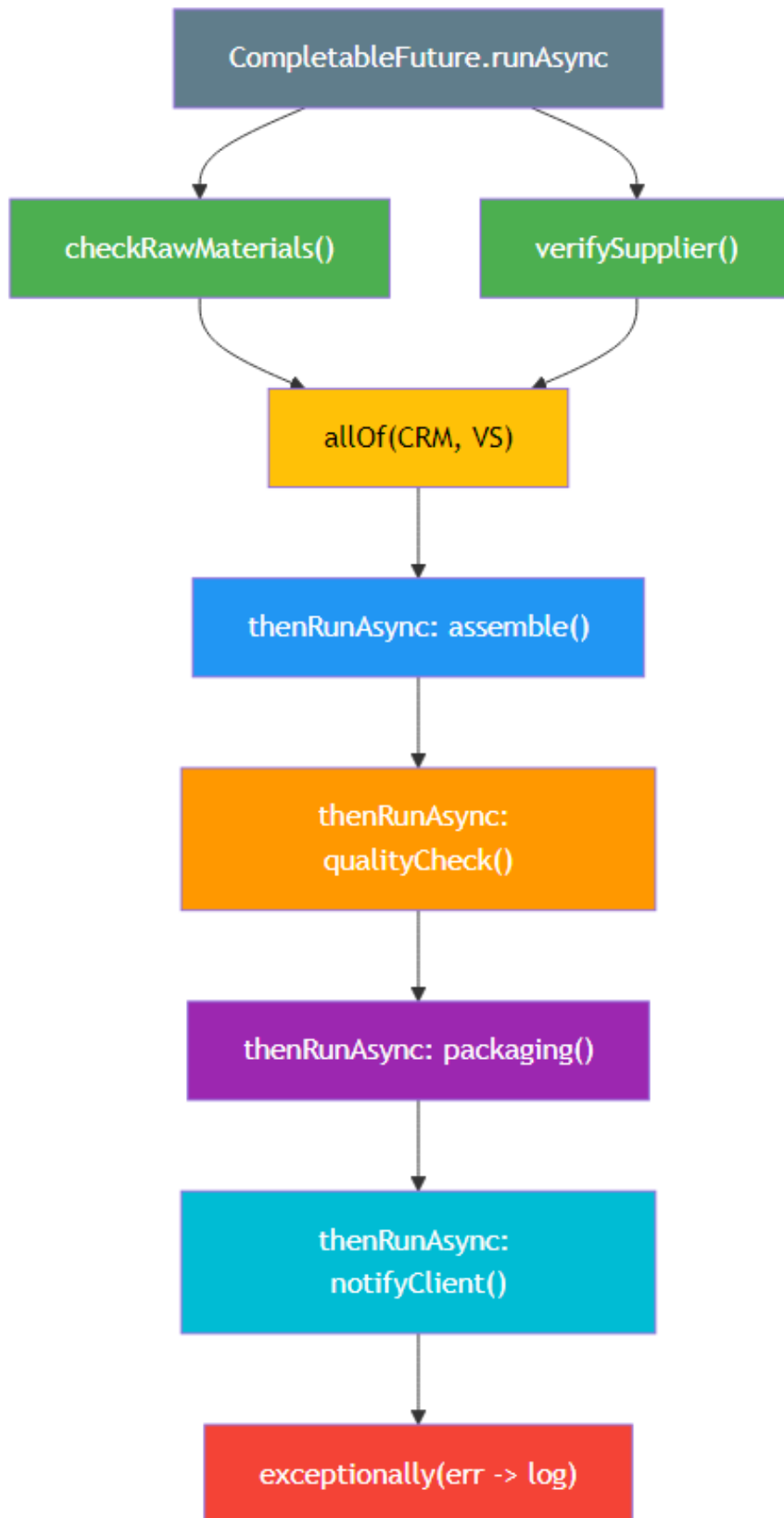
✓ Correction

Listing 23 – Exécution parallèle avec **allOf**

```
1 // Lancer les deux taches en parallele
2 CompletableFuture<Void> checkMaterials =
3     CompletableFuture.runAsync(
4         () -> checkRawMaterials(order), executor);
5
6 CompletableFuture<Void> verifySupplier =
7     CompletableFuture.runAsync(
8         () -> verifySupplier(order), executor);
9
10 // Attendre les deux avant de continuer
11 CompletableFuture<Void> pipeline =
12     CompletableFuture.allOf(checkMaterials, verifySupplier)
13         .thenRunAsync(() -> assemble(order), executor)
14         .thenRunAsync(() -> qualityCheck(order), executor)
15         .thenRunAsync(() -> packaging(order), executor)
16         .thenRunAsync(() -> notifyClient(order), executor);
17
18 pipeline.join(); // attendre la fin
```

Différence entre allOf et thenCombine :

- `allOf(f1, f2, ...)` : attend que **tous** les futures soient complétés. Retourne `CompletableFuture<Void>`. Utile quand on n'a pas besoin des résultats individuels.
- `thenCombine(other, biFunction)` : combine les résultats de **deux** futures en un seul. Utile quand les deux tâches produisent des valeurs à fusionner.

FIGURE 14 – Pipeline `CompletableFuture` avec exécution parallèle

4.6 Partie 6 : Problèmes de concurrence

Questions

1. Que se passe-t-il si plusieurs threads modifient le même objet `Order` ?
2. Identifier les risques : race conditions, data inconsistency.
3. Proposer des solutions.

✓ Correction

1. Si plusieurs threads modifient `Order` simultanément :

- **Race condition** : deux threads lisent le même état, le modifient, et le dernier écrasant le premier
- **Incohérence** : un thread voit un état partiellement mis à jour
- **Corruption** : des structures non thread-safe (ex : `ArrayList`) peuvent être corrompues

2. Risques concrets :

- Le statut de la commande passe de “VALIDATED” à “ASSEMBLED” et “PAID” simultanément
- Le montant total est calculé avec des valeurs intermédiaires incohérentes
- Les items de la commande sont modifiés pendant l'itération $\Rightarrow$ `ConcurrentModificationException`

3. Solutions :

a) **Immutabilité** (recommandé) :

```
1 public record Order(String id, List<Item> items,
2                     double total, OrderStatus status) {
3     public Order withStatus(OrderStatus newStatus) {
4         return new Order(id, items, total, newStatus);
5     }
6 }
```

b) **Synchronisation** :

```
1 public class Order {
2     private final ReentrantLock lock = new ReentrantLock();
3
4     public void updateStatus(OrderStatus status) {
5         lock.lock();
6         try {
7             this.status = status;
8         } finally {
9             lock.unlock();
10        }
11    }
12 }
```

c) **Collections thread-safe** :

```

1 private final List<Item> items =
2     Collections.synchronizedList(new ArrayList<>());
3 // ou
4 private final CopyOnWriteArrayList<Item> items =
5     new CopyOnWriteArrayList<>();

```

d) AtomicReference pour les champs :

```

1 private final AtomicReference<OrderStatus> status =
2     new AtomicReference<>(OrderStatus.CREATED);

```

4.7 Partie 7 : Gestion des performances

Questions

1. Comment éviter la saturation du thread pool ?
2. Que se passe-t-il si toutes les tâches sont bloquantes ?
3. Pourquoi est-il dangereux d'utiliser le `common ForkJoinPool` par défaut ?
4. Proposer une stratégie de monitoring.

✓ Correction

1. Éviter la saturation :

- Utiliser un `ThreadPoolExecutor` avec une **file bornée** (`ArrayBlockingQueue`)
- Configurer une **politique de rejet** : `CallerRunsPolicy` (le thread appelant exécute la tâche), `AbortPolicy` (lance une exception)
- Implémenter du **backpressure** : limiter le nombre de tâches soumises
- Utiliser un `Semaphore` pour limiter la concurrence

2. Si toutes les tâches sont bloquantes (ex : attente I/O), tous les threads du pool sont occupés à ne rien faire ⇒ **thread starvation**. Les nouvelles tâches s'accumulent dans la file. Si une tâche bloquante attend le résultat d'une autre tâche dans le même pool ⇒ **deadlock**.

3. Le `ForkJoinPool.commonPool()` est partagé par toute la JVM :

- **Toutes les `CompletableFuture` sans `executor` l'utilisent**
- Les **parallel streams** l'utilisent
- Une tâche bloquante dans ce pool **affecte toute l'application**
- Sa taille est fixée à `Runtime.getRuntime().availableProcessors() - 1`

⇒ Toujours fournir un **executor dédié** à `CompletableFuture.runAsync`.

4. Stratégie de monitoring :

Listing 24 – Monitoring d'un `ThreadPoolExecutor`

```

1 ThreadPoolExecutor pool = (ThreadPoolExecutor) executor;
2 ScheduledExecutorService monitor =

```



```

3      Executors.newSingleThreadScheduledExecutor();
4
5  monitor.scheduleAtFixedRate(() -> {
6      System.out.println("Pool size: " + pool.getPoolSize());
7      System.out.println("Active: " + pool.getActiveCount());
8      System.out.println("Queue: " + pool.getQueue().size());
9      System.out.println("Completed: " + pool.
10         getCompletedTaskCount());
11 }, 0, 5, TimeUnit.SECONDS);

```

Compléter avec Micrometer/Prometheus pour des métriques exportées.

4.8 Partie 8 : Bonus Architecture

Questions

1. Comment intégrer ce système dans une architecture microservices ou event-driven ?
2. Différence entre parallélisme et asynchronisme ?
3. Quand préférer `CompletableFuture` vs Reactive (WebFlux) ?

✓ Correction

1.

- **Microservices** : chaque étape devient un service indépendant communiquant via REST/gRPC ou messaging (Kafka, RabbitMQ). L'orchestration se fait via un orchestrateur (Saga) ou chorégraphie événementielle.
- **Event-driven** : chaque étape publie un événement à la fin de son traitement. L'étape suivante est déclenchée par cet événement. Avantages : découplage fort, scalabilité indépendante.

2. Comparaison :

	Parallélisme	Asynchronisme
Définition	Exécution simultanée sur plusieurs cœurs	Non-blocage : le thread n'attend pas le résultat
Objectif	Réduire le temps de calcul	Libérer les threads pendant l'attente
Exemple	<code>parallelStream</code> , multi-thread	<code>CompletableFuture</code> , callbacks, reactive
Ressource	CPU	I/O

3.

- **CompletableFuture** : adapté pour un nombre limité de tâches asynchrones, intégration dans du code impératif existant, logique "linéaire" avec quelques branches.
- **Reactive (WebFlux / Project Reactor)** : adapté pour des flux de données continus, très haute concurrence (milliers de requêtes simultanées), backpressure natif, applications entièrement non-bloquantes.

Problème 5 : Conception d'une architecture AWS sécurisée et scalable

Contexte

Une entreprise souhaite migrer son application e-commerce (microservices) vers AWS. L'application permet :

- Consulter des produits
- Passer des commandes
- Gérer les comptes utilisateurs
- Accéder à des contenus statiques (images, vidéos)

Le candidat doit concevoir l'architecture complète : VPC, AZ, Region, ALB, Gateway, Subnets, Security Groups, S3, CloudFront, ECR, ECS, RDS, etc.

Questions

Le candidat doit tracer l'architecture complète en identifiant :

1. Les composants réseau (VPC, subnets, AZ)
2. Les composants de sécurité (Security Groups, ACLs)
3. Les composants de calcul (ECS, Fargate, Lambda)
4. Les composants de stockage (S3, RDS)
5. Les composants de distribution (CloudFront, ALB)
6. L'architecture microservices

5.1 Architecture VPC et Réseau

✓ Correction

Structure réseau :

Composant	AZ1	AZ2	Rôle
Public Subnet	10.0.1.0/24	10.0.2.0/24	ALB, NAT Gateway, Bastion
Private Subnet	10.0.3.0/24	10.0.4.0/24	ECS Fargate Tasks
DB Subnet	10.0.5.0/24	10.0.6.0/24	RDS (Primary + Standby)

Principes :

- **Multi-AZ** : haute disponibilité, le trafic est distribué entre 2 Availability Zones minimum
- **Subnets publics** : contiennent les composants exposés à Internet (ALB, NAT)
- **Subnets privés** : contiennent les services applicatifs, isolés d'Internet
- **DB Subnets** : isolés, accessibles uniquement depuis les subnets privés
- **NAT Gateway** : permet aux services privés d'accéder à Internet (mises à jour, API externes) sans être exposés

5.2 Sécurité – Security Groups

✓ Correction

Security Group	Inbound Rules	Attaché à
SG-ALB	TCP 80/443 depuis 0.0.0.0/0	Application Load Balancer
SG-ECS	TCP 8080 depuis SG-ALB uniquement	ECS Fargate Tasks
SG-RDS	TCP 5432/3306 depuis SG-ECS uniquement	RDS Instances

Principe du moindre privilège : chaque couche n'accepte le trafic que de la couche précédente. Le chaînage des Security Groups garantit qu'aucun accès direct n'est possible depuis Internet vers les bases de données.

5.3 Composants applicatifs

✓ Correction

- **CloudFront** (CDN) : distribue les contenus statiques (images, vidéos, CSS, JS) depuis S3 via un réseau mondial d'edge locations. Réduit la latence et décharge l'ALB.
- **S3** : stockage des assets statiques. Accès uniquement via CloudFront (Origin Access Identity).
- **ALB** (Application Load Balancer) : route le trafic HTTP/HTTPS vers les services ECS. Supporte le routage par path (`/api/products` → Product Service, `/api/orders` → Order Service).
- **ECS Fargate** : exécute les conteneurs Docker sans gérer de serveurs. Auto-scaling basé sur CPU/mémoire/requêtes.
- **ECR** (Elastic Container Registry) : registre privé pour stocker les images Docker des microservices.
- **RDS** (Multi-AZ) : base de données relationnelle avec réplication synchrone. Failover automatique.

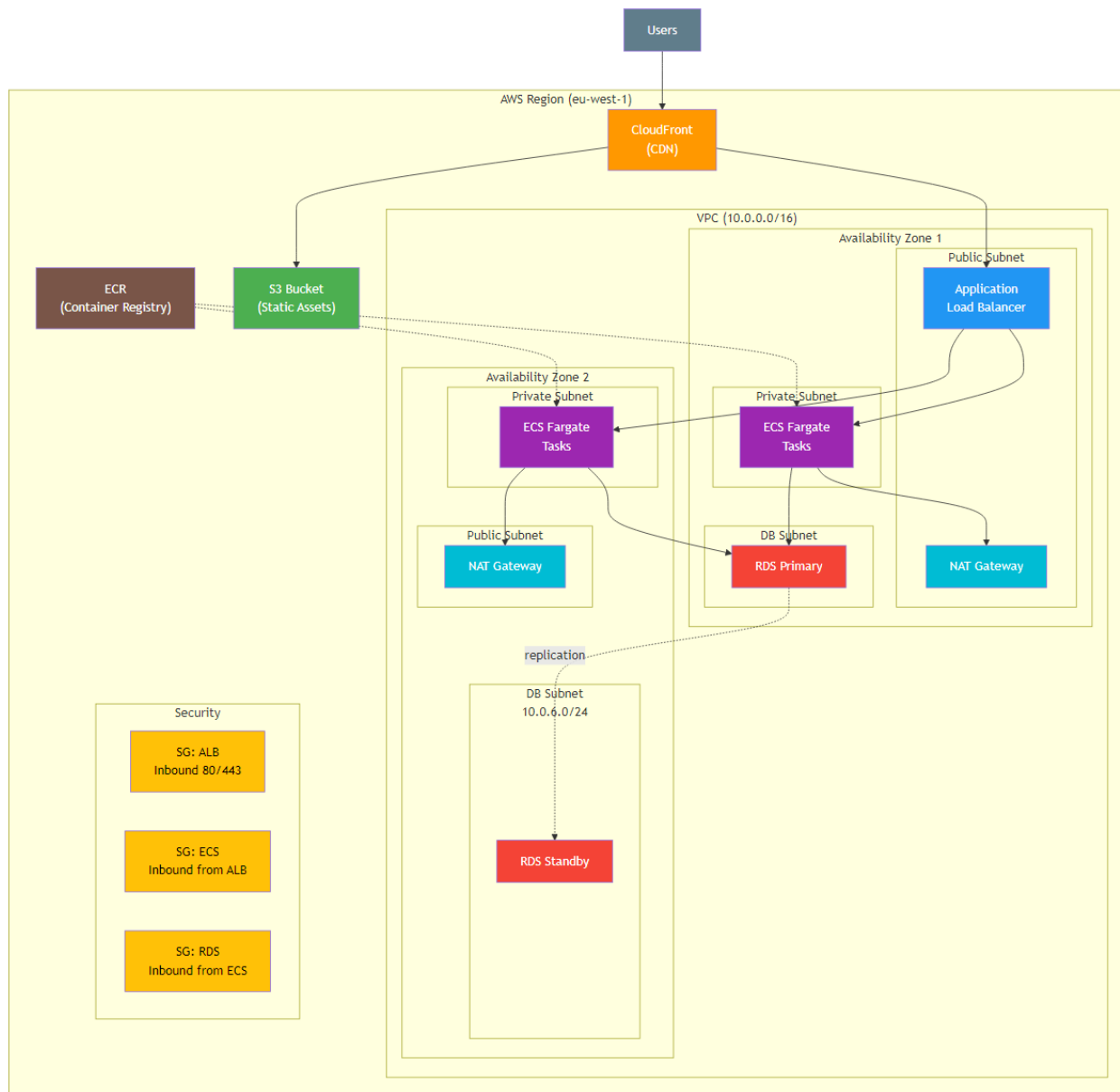


FIGURE 15 – Architecture AWS complète – VPC, Subnets, et composants

5.4 Architecture Microservices

✓ Correction

Chaque microservice est déployé en tant que **service ECS** indépendant :

Service	Responsabilité	Base de données
Product Service	Catalogue produits, recherche	RDS (Products) + Elasti-Cache
Order Service	Gestion commandes, panier	RDS (Orders)
User Service	Authentification, profils	RDS (Users) + Cognito
Payment Service	Traitement paiements	RDS (Payments)
Notification Service	Emails, SMS, Push	Lambda + SES/SNS

Communication inter-services :

- **Synchrone** : via ALB interne (service-to-service) ou API Gateway
- **Asynchrone** : via SQS/SNS pour les événements (commande créée → paiement, notification)
- **Chaque service a sa propre BDD** (pattern Database per Service)

Services AWS complémentaires :

- **API Gateway** : point d'entrée unique, rate limiting, authentification
- **Cognito** : gestion d'identité et authentification utilisateurs
- **ElastiCache (Redis)** : cache pour le catalogue produits (réduction latence)
- **CloudWatch** : monitoring, logs centralisés, alertes
- **SES/SNS** : envoi d'emails et notifications push

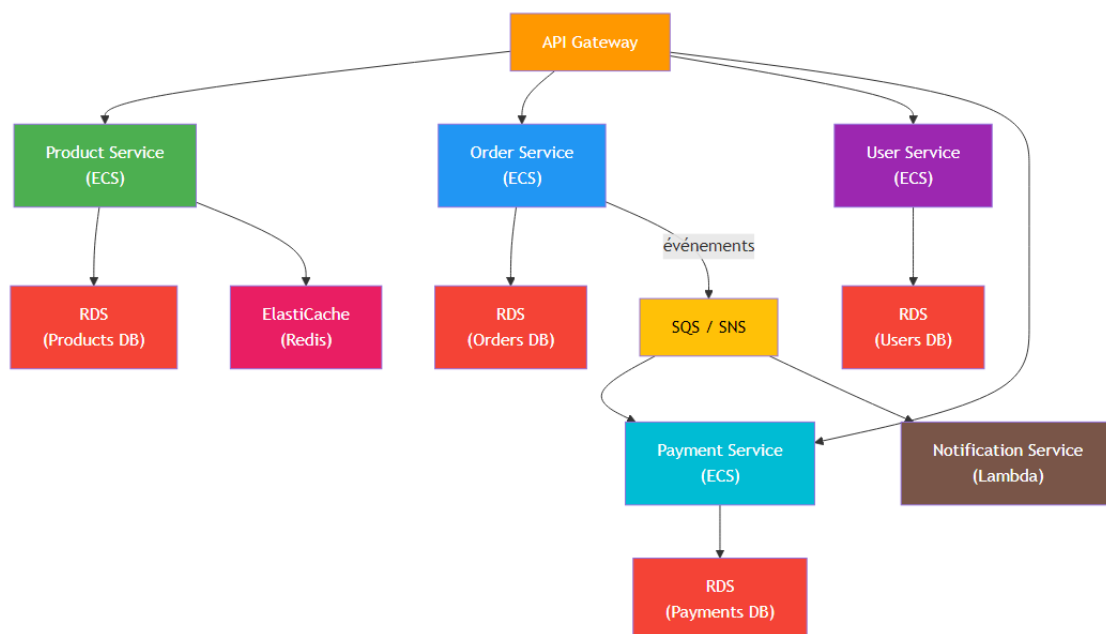


FIGURE 16 – Architecture Microservices sur AWS

5.5 Scalabilité et Haute Disponibilité

✓ Correction

- **Auto Scaling ECS** : augmente/réduit le nombre de tâches Fargate selon la charge (CPU, mémoire, nombre de requêtes)
- **Multi-AZ** : tous les composants critiques sont répliqués sur au moins 2 Availability Zones
- **RDS Multi-AZ** : failover automatique en < 60 secondes
- **CloudFront** : CDN mondial avec cache distribué
- **ALB** : distribue la charge et effectue des health checks réguliers
- **S3** : durabilité de 99,999999999% (11 neufs)

Stratégie de déploiement :

- **Blue/Green** ou **Rolling Update** via ECS
- **CI/CD** : CodePipeline + CodeBuild + ECR + ECS
- **Infrastructure as Code** : CloudFormation ou Terraform